

ALL PROGRAMMING LANGUAGES

*A Comprehensive Taxonomy, Historical Synthesis, Comparative Framework,
and Research Agenda*

SAMUELSON G

Independent Researcher

Email: gsamuelsonguna@gmail.com

ORCID: 0009-0005-8744-8178

Review Article / Structured Narrative Synthesis

Manuscript date: 13 July 2026

Abstract

Programming languages constitute a continually expanding family of formal and practical systems for expressing computation. A literal survey of every language is impossible because the boundary between programming, data, markup, configuration, query, hardware-description, visual, and domain-specific languages is porous, historical records are incomplete, and new languages and dialects appear continuously. This paper therefore develops a comprehensive framework rather than an exhaustive list. It combines a structured narrative review of foundational papers, standards, language reports, and current official specifications with a representative comparative corpus spanning imperative, object-oriented, functional, logic, concurrent, scripting, scientific, systems, educational, and domain-specific traditions. The synthesis models a language as a seven-dimensional socio-technical system comprising syntax, semantics, type discipline, memory/resource model, evaluation strategy, concurrency model, and ecosystem/interoperability. It traces major historical lineages; compares representative languages by abstraction mechanisms, execution model, safety, performance control, portability, and domain fit; and explains why no universal language can dominate all workloads. The paper further proposes a weighted decision model for language selection and identifies research priorities in safe systems programming, gradual and dependent typing, effect systems, heterogeneous computing, verified compilation, energy-aware programming, multilingual software, end-user programming, and AI-assisted development. The central conclusion is that programming-language progress is best understood not as a sequence of replacements but as cumulative recombination: modern languages inherit, refine, and operationalize ideas developed across many earlier traditions.

Keywords: programming languages; language design; paradigms; type systems; compilers; interpreters; memory safety; concurrency; domain-specific languages; software ecosystems

Highlights

- Defines a publishable scope for the phrase “all programming languages” without claiming an impossible exhaustive census.
- Introduces a seven-dimensional model that integrates formal language properties with runtime and ecosystem considerations.
- Provides a historical timeline, paradigm taxonomy, implementation model, comparative tables, and an evidence-based selection workflow.
- Shows that language choice is a constrained multi-objective decision rather than a universal popularity contest.
- Sets out a research agenda for safety, verification, heterogeneous hardware, AI-assisted programming, and multilingual systems.

1. Introduction

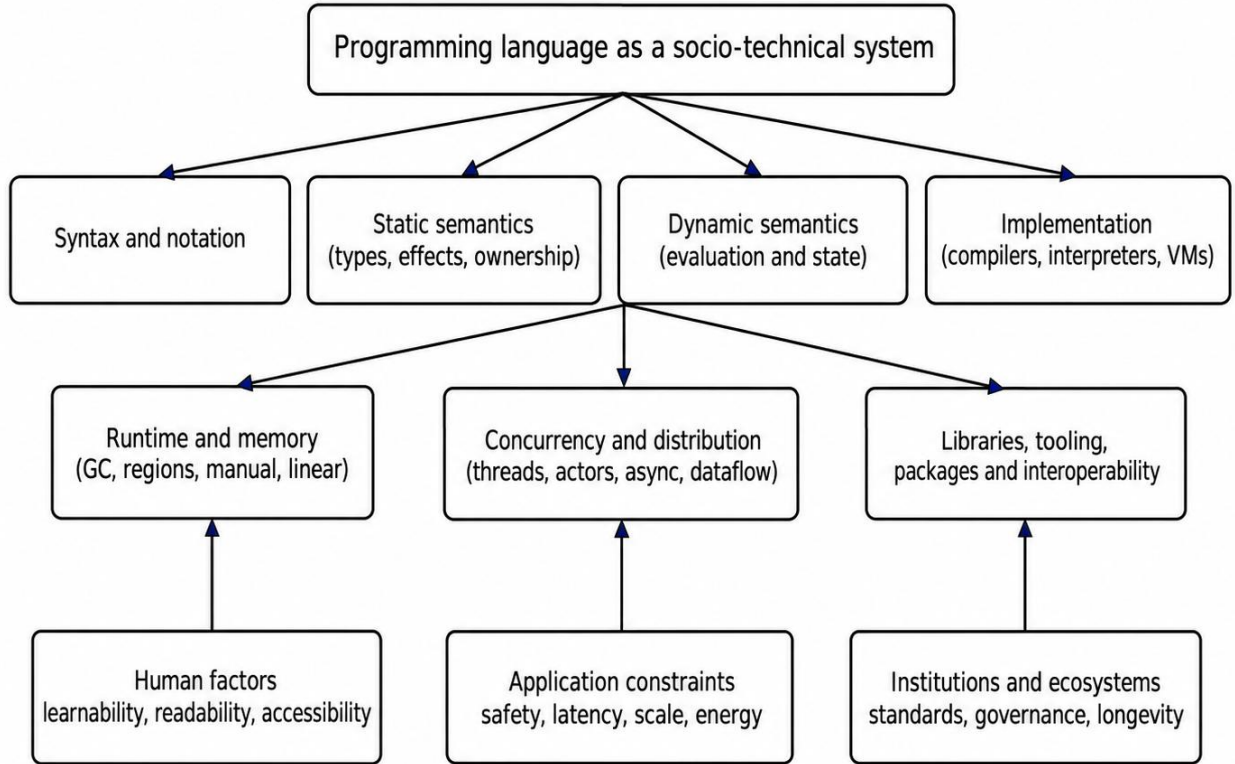
Programming languages are engineered notations through which humans specify computations, coordinate machines, and organize knowledge. They range from direct encodings of processor operations to declarative systems that state relationships or desired

results. Between these extremes lie general-purpose languages, scripting languages, query languages, hardware-description languages, visual environments, educational notations, and thousands of specialized languages embedded in scientific, industrial, and administrative practice.

The title “All Programming Languages” poses a methodological problem. There is no universally accepted definition of what counts as a distinct language rather than a version, dialect, macro system, notation, or embedded domain-specific language. A July 2026 snapshot of GitHub Linguist contained 815 language entries, of which 546 were classified as programming languages; however, Linguist is an operational source-code classification system, not a historical or theoretical census [50]. This paper accordingly uses “all” to mean the full design space and major traditions, not every named artifact.

The modern language landscape rests on a small number of durable ideas. FORTRAN demonstrated that automatic translation could combine concise problem statements with competitive machine performance [1], Lisp made symbolic expressions and recursive functions central objects of computation [3], ALGOL established influential block structure and precise syntax description [4], and C linked portable high-level programming to systems implementation [10]. Object orientation, logic programming, functional programming, type inference, garbage collection, virtual machines, actors, gradual typing, ownership, and domain-specific abstraction subsequently enlarged the design space rather than collapsing it.

This review has four objectives: (1) define a stable taxonomy capable of positioning old and new languages; (2) synthesize the historical development of major language families; (3) compare representative languages without reducing them to a single rank; and (4) propose a transparent method for selecting languages under real project constraints. The result is intended as a reference for researchers, educators, architects, and practitioners.



Author's synthesis. The model treats a programming language as both a formal system and an ecosystem.

Figure 1. Multidimensional model used in this review. A programming language is treated as both a formal system and a socio-technical ecosystem. Author's synthesis.

2. Scope, Definitions, and Review Method

2.1 Operational definition

A programming language is defined here as a notation with sufficiently specified syntax and semantics to direct, describe, or constrain computation by a machine or computational agent. This inclusive definition covers conventional executable languages, query and rule languages, hardware-description languages, shader languages, and executable configuration or modeling languages. Pure markup and prose formats are excluded unless they contain an execution or transformation semantics.

$$L = \langle S, Sem, T, M, E, C, X \rangle \quad (1)$$

Equation (1) represents a language L by seven dimensions: syntax S ; static and dynamic semantics Sem ; type discipline T ; memory and resource model M ; evaluation and execution strategy E ; concurrency and communication model C ; and ecosystem, tooling, and interoperability X . The tuple is descriptive rather than axiomatic: some languages leave one or more dimensions to implementations or host environments.

2.2 Review design

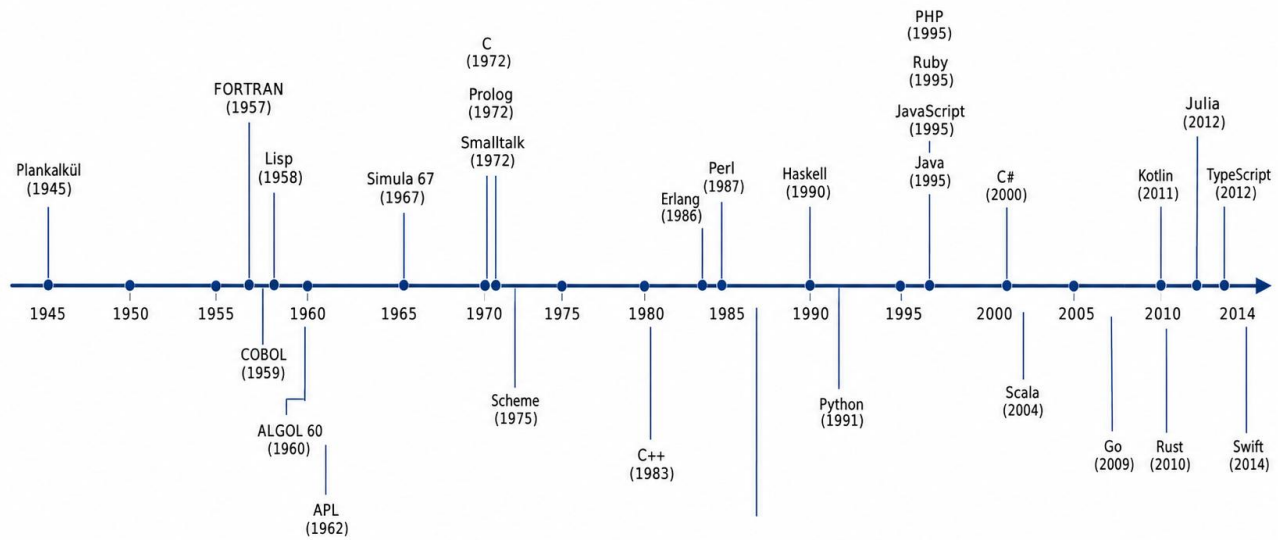
The study is a structured narrative synthesis. Sources were prioritized in the following order: original language papers and historical retrospectives; international standards and official language specifications; peer-reviewed work on semantics, type systems, safety, and implementation; and maintained official documentation. Popularity rankings and unsourced comparison websites were not used as evidence of technical superiority.

The representative corpus was selected by maximum conceptual variation. It includes historically foundational languages, major general-purpose ecosystems, languages that introduced influential mechanisms, and domain-specific exemplars. Inclusion does not imply endorsement, and omission does not imply insignificance. The comparison emphasizes design choices and consequences rather than transient market share.

2.3 Limitations

- The corpus is representative rather than exhaustive; thousands of experimental, educational, esoteric, proprietary, and domain-specific languages are not individually tabulated.
- Language properties vary by version and implementation. The table records dominant or standard behavior, not every extension.
- Performance cannot be inferred from language names alone; algorithms, compilers, runtimes, libraries, hardware, and programmer expertise are often decisive.
- Ecosystem maturity and institutional support change over time. Current specification references are dated and should be rechecked at submission.
- Several classifications are cross-cutting. A language may be imperative, functional, object-oriented, concurrent, and domain-specific at the same time.

3. Historical Evolution



Dates are approximate first publication or public introduction. The timeline is illustrative, not exhaustive.

Figure 2. Selected milestones in programming-language evolution. Dates indicate approximate first publication or public introduction; the timeline is illustrative and not exhaustive. Author's synthesis.

3.1 From machine notation to automatic programming

Early programming was tightly coupled to machine instruction sets. Plankalkül anticipated structured data and high-level notation, but FORTRAN became the decisive industrial demonstration that a compiler could translate mathematical formulations into efficient machine programs [1,2]. The project also established optimization as a central compiler responsibility. COBOL pursued a different goal: readable business data processing and institutional portability. ALGOL 60 contributed lexical scope, block structure, recursive procedures, and a publication language whose grammar could be described systematically [4].

These developments separated program meaning from a particular machine while simultaneously creating new problems: precise specification, compiler correctness, portability, and the tension between convenience and predictable cost. Those tensions remain visible in present debates about managed runtimes, zero-cost abstractions, undefined behavior, and ahead-of-time versus dynamic optimization.

3.2 Symbolic, functional, and logical traditions

McCarthy's Lisp described symbolic expressions, recursive functions, conditional expressions, and a representation in which code and data share a common structural form [3]. Later Lisp and Scheme systems made macros, interactive development, lexical scope, closures, and metaprogramming central. The functional tradition developed stronger mathematical foundations through lambda calculus, denotational semantics, polymorphic type systems, type classes, and controlled effects. Haskell consolidated lazy, purely functional programming with static type inference and type classes [24,25].

Logic programming approached computation as proof search over relations. Prolog used unification and backtracking to make symbolic relations executable. Constraint logic programming, deductive databases, answer-set programming, and relational programming extended this view. These traditions show that programs need not be organized primarily as sequences of state-changing commands.

3.3 Structured, object-oriented, and systems programming

Structured programming promoted disciplined control flow and reasoning about program states [8,9]. C emerged as a compact systems language whose abstractions mapped closely enough to hardware for operating-system implementation while remaining portable across machines [10]. Its success also exposed enduring risks: manual memory management, unrestricted pointer arithmetic, and behaviors whose meaning depends on implementation or is deliberately undefined.

Simula introduced classes, objects, inheritance, and virtual procedures in the context of simulation. Smalltalk then treated message-passing objects and a live graphical environment as a unified computing medium [11]. C++, Java, C#, Objective-C, Python, Ruby, JavaScript, and many others adopted different mixtures of classes, prototypes, interfaces, generics, reflection, and managed runtimes. Object orientation became one major abstraction family, but contemporary practice usually combines it with functional and procedural techniques.

3.4 Scripting, web, managed runtimes, and multi-paradigm design

Unix shells, awk, Perl, Tcl, Python, Ruby, PHP, and JavaScript emphasized rapid composition, text processing, dynamic data, and interactive feedback. Java popularized a strongly specified virtual machine and bytecode ecosystem for portable managed execution. ECMAScript evolved from browser scripting into a standardized general-purpose language with a yearly specification process; the 2026 standard is the seventeenth edition [18]. Python's reference describes its syntax and core semantics while acknowledging that some details are intentionally less formal than a complete mathematical specification [19].

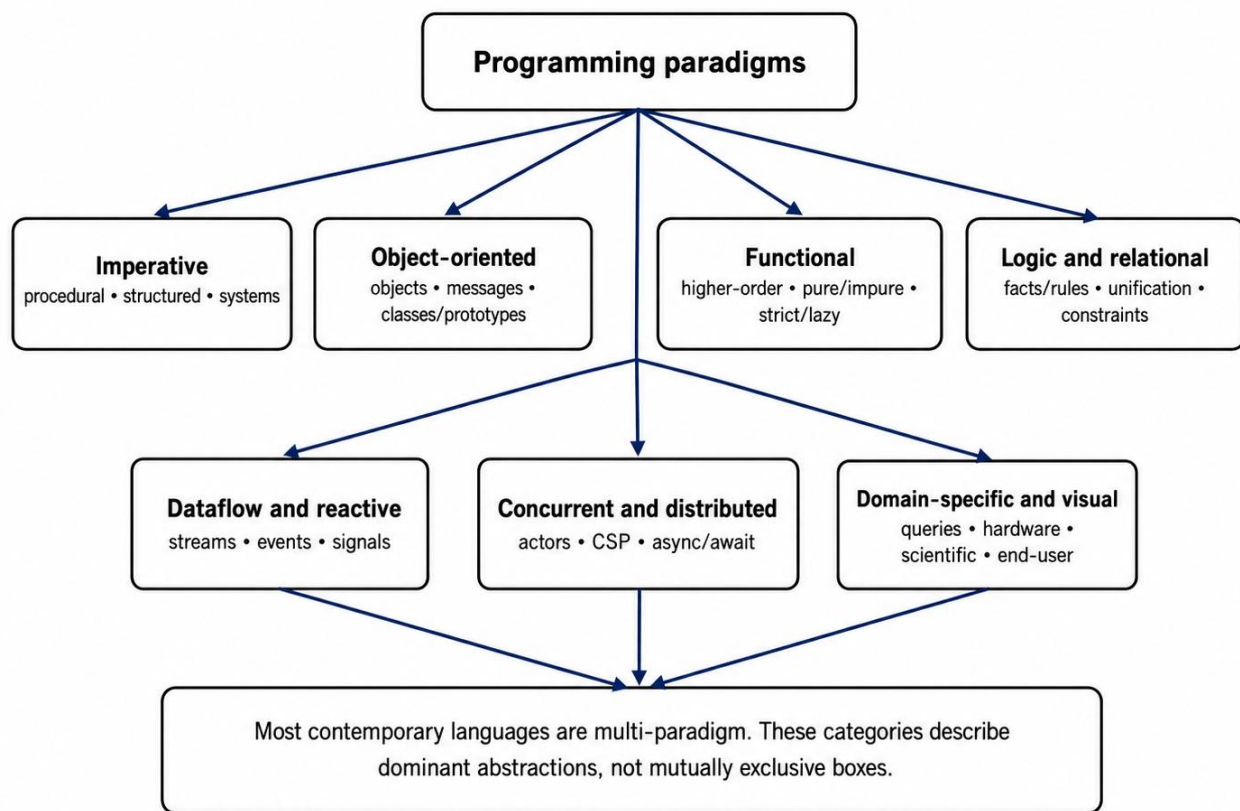
Modern languages are increasingly multi-paradigm. Scala combines object-oriented and functional concepts; Kotlin combines nominal object orientation with functional idioms and null-safety; TypeScript layers gradual structural typing over JavaScript; Julia combines

dynamic multiple dispatch with specialization for scientific computing; and Swift integrates value semantics, protocols, automatic reference counting, and systems-level interoperability.

3.5 Safety, concurrency, and resource-aware design

The growth of networked and concurrent software shifted language design toward isolation, asynchronous control, and data-race prevention. Erlang’s actor model and supervision strategies support fault-tolerant distributed systems [26]. Go combines lightweight goroutines with channels and a compact language specification [20]. Rust uses ownership, borrowing, lifetimes, and an explicit unsafe boundary to target memory safety without requiring tracing garbage collection; formal work such as RustBelt studies the semantic foundations needed to justify unsafe library abstractions [23].

4. A Taxonomy of Programming Languages



Author’s synthesis. A single language may occupy several categories simultaneously.

Figure 3. Cross-cutting taxonomy of programming paradigms. Most modern languages occupy multiple categories. Author’s synthesis.

Table 1. Major language categories and representative examples

Category	Defining emphasis	Representative languages	Typical uses
Imperative / procedural	Commands update state; control flow sequences actions	FORTRAN, C, Pascal, Ada	Systems, numerical, embedded, general-purpose
Object-oriented	Objects encapsulate state and behavior; interaction via calls or messages	Simula, Smalltalk, C++, Java, C#	Large applications, frameworks, GUIs, enterprise systems
Functional	Functions and expressions are primary; often favors immutability	Lisp, Scheme, ML, Haskell, F#, Clojure	Compilers, formal methods, data transformation, concurrent systems
Logic / relational	Programs state facts, rules, relations, or constraints	Prolog, Datalog, Mercury, miniKanren	Knowledge representation, analysis, planning, databases
Scripting / glue	Rapid composition, dynamic values, automation, text and system interaction	Shell, Perl, Python, Ruby, Lua, Tcl	Automation, web, testing, data preparation, embedding
Concurrent / distributed	Concurrency and communication are first-class design concerns	Erlang, Elixir, Go, Pony, Chapel	Services, telecom, parallel and distributed systems
Array / data-parallel	Whole-array operations and bulk transformations	APL, J, K, MATLAB, R, Julia	Scientific, statistical, financial, exploratory computing
Domain-specific	Notation optimized for a constrained problem domain	SQL, Verilog, VHDL, GLSL, R, regex, build languages	Databases, hardware, graphics, modeling, configuration
Visual / block-based	Programs expressed through diagrams, blocks, or spatial composition	Scratch, LabVIEW, Simulink	Education, instrumentation, control, end-user programming
Esoteric / experimental	Explores minimalism, unusual semantics, pedagogy, or artistic constraints	Brainfuck, Befunge, Piet	Research, recreation, language theory, pedagogy

4.1 Paradigm is not identity

Paradigm labels are useful summaries but poor substitutes for specification-level analysis. Java is commonly classified as object-oriented, yet it includes procedural control flow, generic types, exceptions, reflection, lambda expressions, and concurrent libraries. Python

supports procedural, object-oriented, functional, reflective, and metaprogramming styles. C++ spans low-level resource control, generic metaprogramming, object orientation, functional idioms, compile-time computation, and domain-specific library construction. The appropriate unit of comparison is therefore a bundle of mechanisms and constraints.

4.2 Type disciplines

Type systems classify values and expressions and constrain composition. Static typing checks many properties before execution; dynamic typing checks operations at run time; gradual systems allow typed and untyped regions to interact. Nominal systems relate types through declarations, while structural systems relate them through available operations or fields. Parametric polymorphism, subtyping, ad hoc polymorphism, type classes, traits, dependent types, refinement types, linear types, affine types, and effect systems provide different balances among expressiveness, inference, decidability, and error reporting [13,14,32].

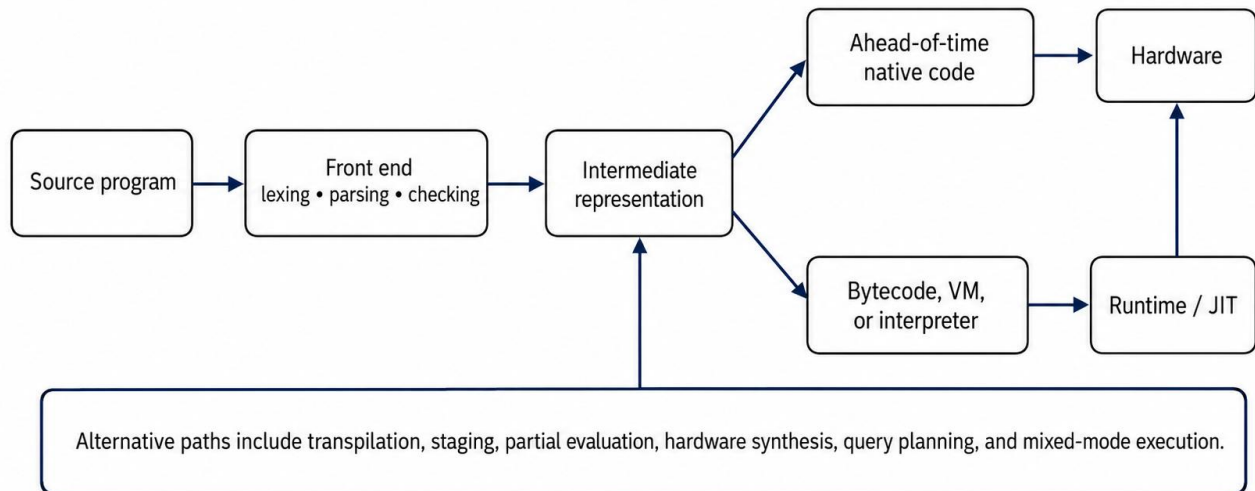
No single point on this design space is always superior. Stronger static guarantees can reduce classes of errors and improve tools, but may increase annotation burden, compilation complexity, or conceptual difficulty. Dynamic systems can support rapid evolution and reflective patterns, but shift failures to tests or execution. Gradual typing attempts staged adoption, although boundary semantics and performance may become complicated.

4.3 Memory and resource models

Memory management ranges from explicit allocation and deallocation to tracing garbage collection, reference counting, regions, arena allocation, ownership, borrow checking, and linear resource tracking. A language may standardize a model or leave it to an implementation. Memory policy interacts with latency, throughput, fragmentation, pause behavior, predictability, foreign-function interfaces, and safety. Rust’s ownership approach is distinctive because many invalid aliasing and lifetime patterns are rejected statically, while controlled unsafe code remains available for low-level abstractions [23].

4.4 Evaluation and execution

Languages may use eager or lazy evaluation, deterministic or nondeterministic choice, interpretation, bytecode, virtual machines, just-in-time compilation, ahead-of-time compilation, source-to-source translation, or hardware synthesis. These categories are implementation paths rather than intrinsic quality rankings. A language with a formal semantics can have multiple implementations; a single implementation can mix interpretation, profiling, tiered compilation, specialization, and native extensions.



Author's synthesis. Real systems often combine interpretation, profiling, JIT compilation, native extensions, and ahead-of-time compilation.

Figure 4. Common implementation paths from source to execution. Real systems often combine several paths. Author's synthesis.

5. Comparative Analysis of Representative Languages

Table 2. Comparative profile of representative programming languages

Language	Origin	Dominant paradigm	Typing	Memory/resource model	Execution	Characteristic domains
FORTRAN	1957	Imperative, array	Static	Manual / implementation	AOT native	Numerical and HPC
COBOL	1959	Imperative, business	Static	Managed by runtime / records	AOT / VM	Enterprise transactions
Lisp / Common Lisp	1958 / 1984	Functional, symbolic, OO	Dynamic	Garbage collected	Native / bytecode / interpreter	AI, symbolic, extensible systems
ALGOL 60	1960	Structured procedural	Static	Stack-oriented	AOT	Language research, algorithms
APL	1962	Array, functional	Dynamic	Managed	Interpreter / JIT	Mathematics, analytics

Simula	1967	Object-oriented, simulation	Static	Managed / implementation	AOT	Simulation, OO foundations
C	1972	Procedural, systems	Static	Manual	AOT native	OS, embedded, infrastructure
Prolog	1972	Logic, relational	Dynamic / logical terms	Managed	WAM / interpreter / native	Rules, search, symbolic reasoning
Smalltalk	1972	Object-oriented, reflective	Dynamic	Garbage collected	VM / JIT	Live systems, GUIs, education
Scheme	1975	Functional, minimalist	Dynamic	Garbage collected	Interpreter / native	Education, language research
C++	1983	Multi-paradigm, systems	Static	Manual / RAII / smart pointers	AOT native	Performance-critical systems
Erlang	1986	Functional, actor	Dynamic	Per-process GC	BEAM VM	Fault-tolerant distributed systems
Perl	1987	Scripting, text	Dynamic	Managed	Interpreter	Automation, text, legacy web
Haskell	1990	Pure functional, lazy	Static inferred	Garbage collected	Native / interpreter	Research, compilers, high-assurance software
Python	1991	Multi-paradigm scripting	Dynamic + optional annotations	Garbage collected / refcount in CPython	Bytecode / interpreter / JIT variants	Data, web, automation, education
R	1993	Statistical, functional, vector	Dynamic	Garbage collected	Interpreter / JIT components	Statistics, data science
Java	1995	OO, imperative, functional features	Static	Garbage collected	JVM bytecode / JIT / AOT	Enterprise, services, Android ecosystem

JavaScript / ECMAScript	1995	Prototype OO, functional, event-driven	Dynamic	Garbage collected	JIT / interpreter	Web, servers, applications
Ruby	1995	OO, scripting, functional features	Dynamic	Garbage collected	VM / JIT	Web, automation, DSLs
PHP	1995	Web scripting, multi-paradigm	Dynamic with type features	Managed	VM / JIT	Server-side web
C#	2000	OO, functional, component	Static	Garbage collected	.NET IL / JIT / AOT	Enterprise, desktop, games, services
Scala	2004	OO + functional	Static inferred	Garbage collected	JVM / native / JS	Data platforms, services, FP
Go	2009	Imperative, concurrent	Static	Garbage collected	AOT native	Cloud services, networking, tools
Rust	2010	Systems, functional influences	Static inferred	Ownership / borrowing	AOT native	Safe systems, infrastructure, embedded
Kotlin	2011	OO + functional	Static inferred	Managed / target-dependent	JVM / native / JS	Android, services, multiplatform
Julia	2012	Multiple dispatch, scientific	Dynamic with specialization	Garbage collected	JIT native	Scientific and numerical computing
TypeScript	2012	Typed JavaScript, multi-paradigm	Gradual structural	Host-managed	Transpiled to JavaScript	Large web and Node.js systems
Swift	2014	Protocol-oriented, OO, functional	Static inferred	ARC + ownership features	AOT / JIT contexts	Apple platforms, systems

		1				
SQL	1974	Declarative relational	Schema/type dependent	Database-managed	Query optimizer / execution engine	Data definition and querying
Verilog / VHDL	1984 / 1985	Hardware description, concurrent	Static	Signal / hardware semantics	Simulation / synthesis	Digital hardware design

5.1 Safety and correctness

Safety is multidimensional. Type safety concerns whether operations are applied to compatible values; memory safety concerns invalid access, lifetime, and aliasing; concurrency safety concerns races and synchronization; capability safety concerns authority; and domain safety concerns invariants such as units, ranges, protocols, or information flow. A language may be strong in one dimension and weak in another. Java eliminates many raw-pointer errors but cannot by itself guarantee absence of data races or logical faults. C offers precise layout and interoperability but requires disciplined tooling and engineering to control memory hazards. Rust moves many ownership and aliasing obligations into static analysis, but unsafe code, foreign interfaces, logic errors, and resource exhaustion remain possible.

5.2 Performance and predictability

Language-level mechanisms affect optimization opportunities, yet benchmark results cannot be generalized without workload, compiler, runtime, and measurement context. Ahead-of-time native languages often provide strong control over layout and latency; JIT systems can exploit run-time profiles; array languages can expose bulk operations; managed runtimes can improve allocation throughput and locality while introducing pause or footprint concerns. High-level abstractions can be either expensive or optimized away. The relevant question is not whether a language is “fast,” but whether its implementation model permits the required performance envelope with acceptable engineering cost.

5.3 Productivity, maintainability, and learnability

Concise syntax is not equivalent to maintainability. Useful indicators include locality of reasoning, explicitness of effects, quality of diagnostics, refactoring support, module boundaries, testability, API stability, build reproducibility, and the ease with which new maintainers can infer intent. Dynamic languages can reduce ceremony and speed exploratory work; static languages can make large-scale changes safer through machine-checked

interfaces. The balance depends on program lifetime, team size, domain volatility, and the cost of failure.

5.4 Ecosystems and interoperability

A language is rarely adopted in isolation. Libraries, package registries, compilers, debuggers, editors, deployment targets, observability, security processes, governance, licensing, and available expertise may outweigh isolated language features. Interoperability often determines long-term viability: C ABIs, JVM and .NET ecosystems, JavaScript hosts, Python extension modules, WebAssembly targets, and foreign-function interfaces allow languages to coexist. Consequently, multilingual architecture is normal rather than exceptional.

Table 3. Recurring programming-language design trade-offs

Design choice	Potential advantages	Potential costs	Often suitable when
Static guarantees	Earlier error detection, stronger tooling, optimization information	Annotation or inference complexity; restrictive abstractions	Critical or long-lived code with stable interfaces
Dynamic flexibility	Rapid iteration, reflection, flexible data, concise prototypes	Later failures, weaker whole-program guarantees	Exploration, scripting, highly dynamic domains
Manual memory	Control over layout, lifetime, allocation, and latency	Use-after-free, leaks, aliasing errors, higher review burden	Kernels, embedded, specialized runtimes
Tracing garbage collection	Simple object lifetimes, safe reclamation, high allocation throughput	Pauses, footprint, reduced deterministic control	Managed services, applications, interactive systems
Ownership / linearity	Static resource safety without general tracing GC	Learning curve, restrictive alias patterns, compiler complexity	Safe systems and resource-sensitive code
Virtual machines / JIT	Portability, dynamic optimization, rich runtime services	Warm-up, runtime footprint, deployment complexity	Long-running services and managed platforms
Ahead-of-time native	Fast startup, predictable deployment, hardware control	Less run-time specialization; platform-specific artifacts	CLI tools, embedded, mobile, systems
Language minimalism	Small specification, easier tooling and teaching	More burden on libraries and conventions	Infrastructure, education, constrained teams

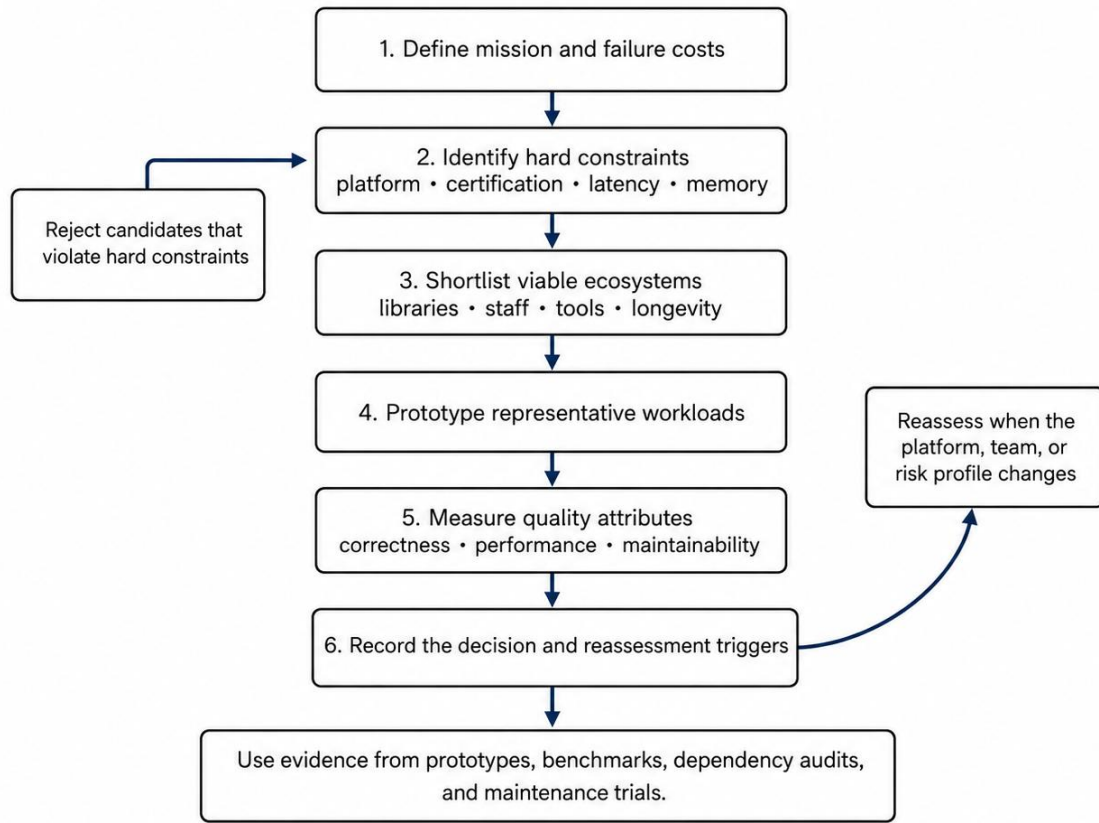
Feature richness	Expressive domain modeling and abstraction	Interaction complexity, style fragmentation	Large ecosystems with expert tooling
DSL specialization	High domain expressiveness and validation	Narrow scope, integration and maintenance costs	Stable, high-value domains

6. Language Selection as a Multi-Objective Decision

Language selection should begin with requirements and failure costs, not familiarity or trend. Hard constraints eliminate candidates that cannot target the platform, satisfy certification rules, meet latency or memory bounds, interoperate with required systems, or support essential libraries. Remaining candidates can then be evaluated using weighted evidence.

$$\text{Score}(L) = \sum_i w_i s_i(L) - \sum_j \lambda_j r_j(L) \quad (2)$$

In Equation (2), $s_i(L)$ is the evidence-based score of language L on benefit criterion i , w_i is the project-specific importance of that criterion, $r_j(L)$ is a risk score, and λ_j is the organization's risk sensitivity. Scores should be supported by prototypes, measurements, maintenance experiments, ecosystem audits, and security review rather than intuition alone. The model is not intended to manufacture false precision; its purpose is to make assumptions visible.



Author's synthesis. Language choice is treated as a constrained multi-objective engineering decision.

Figure 5. Evidence-based language-selection workflow. The process begins with failure costs and hard constraints, then uses prototypes and measurements. Author's synthesis.

Table 4. Suggested evaluation rubric for language selection

Criterion	Questions	Evidence
Correctness and safety	Type, memory, concurrency, capability, domain invariants	Compiler experiments; formal guarantees; defect history; static analysis
Performance	Throughput, latency, startup, footprint, energy	Representative benchmarks on target hardware; profiling
Reliability	Fault isolation, recovery, determinism, deployment behavior	Failure injection; soak testing; runtime analysis
Maintainability	Modularity, readability, refactoring, API evolution	Change tasks performed by representative developers
Ecosystem	Libraries, package health, tooling, observability, security	Dependency audit; maintainer activity; vulnerability process
Interoperability	FFI, protocols, data formats, build	End-to-end integration prototype

	integration	
Team capability	Existing expertise, hiring, training, review quality	Skills inventory; onboarding trial
Governance and longevity	Standardization, stewardship, compatibility policy	Roadmap and release-history review
Compliance	Certification, licensing, export, accessibility, privacy	Legal and assurance review
Strategic flexibility	Portability, replaceability, open implementations	Exit plan; alternative implementation test

7. Domain-Specific Perspectives

7.1 Systems and embedded software

Systems software prioritizes predictable layout, low overhead, interoperability, direct hardware access, and controlled allocation. C and C++ remain deeply embedded in operating systems, firmware, devices, and performance libraries. Rust offers a newer compromise that preserves native compilation and explicit resource control while statically enforcing many memory-safety rules. Ada and SPARK illustrate another path: strong typing, contracts, tasking, and verification-oriented profiles for high-integrity systems.

7.2 Scientific, numerical, and data-intensive computing

FORTTRAN’s array-oriented numerical model remains important in high-performance computing. C and C++ supply infrastructure and accelerator interfaces; Python coordinates libraries written in lower-level languages; R emphasizes statistical analysis; MATLAB and Julia provide interactive numerical environments; SQL and dataflow systems operate over large data collections. In this domain, the effective language is often a stack: a high-level orchestration language, optimized libraries, generated kernels, and device-specific code.

7.3 Web, mobile, and enterprise applications

Web software is inherently multilingual: HTML and CSS define presentation, ECMAScript defines browser computation, server systems use Java, C#, JavaScript/TypeScript, Python, PHP, Ruby, Go, Rust, or other languages, and SQL or specialized query systems manage data. Mobile development adds Swift and Objective-C on Apple platforms and Kotlin or Java on Android. Enterprise longevity makes compatibility, observability, deployment tooling, library governance, and staffing at least as important as syntax.

7.4 Education and end-user programming

Educational languages expose different conceptual models. Logo foregrounds procedures and exploratory construction; Scratch uses blocks to prevent many syntax errors; Scheme supports small semantic cores; Python offers readable syntax and immediate feedback; visual systems such as LabVIEW and Simulink align programs with domain diagrams. No single teaching language is neutral: each selects concepts to make visible and others to postpone.

7.5 Hardware, graphics, and specialized computation

Hardware-description languages describe circuits and concurrent signal behavior rather than ordinary instruction sequences. Shader languages express massively parallel graphics computations. SQL describes relational results while an optimizer chooses execution plans. Regular-expression engines, build languages, configuration languages, theorem provers, and probabilistic programming systems likewise demonstrate that domain-specific semantics can deliver greater leverage than general-purpose syntax when the domain is stable and valuable.

8. Emerging Directions and Open Research Problems

Table 5. Research agenda for programming languages

Area	Core problem	Promising directions
Safe low-level programming	Combine predictable performance, interoperability, and verifiable memory/resource safety	Ownership, capabilities, linearity, verified unsafe abstractions
Gradual and dependent typing	Increase guarantees without making development or inference impractical	Better boundaries, error localization, proof automation
Effect and resource systems	Track I/O, mutation, exceptions, protocols, energy, and quantitative resources	Usable effect polymorphism and tooling
Concurrency and distribution	Prevent races and protocol errors while tolerating partial failure	Session types, actors, structured concurrency, deterministic parallelism
Verified compilation	Preserve source meaning through optimizing transformations and heterogeneous targets	Proof-producing compilers, translation validation
Heterogeneous hardware	Program CPUs, GPUs, accelerators, FPGAs, and near-data devices coherently	Portable intermediate representations and cost models
Energy and sustainability	Make energy, carbon, and hardware utilization visible to programmers	Energy-aware types, profilers, scheduling semantics
Multilingual systems	Reason across FFIs, generated code, serialization, and independent runtimes	Cross-language contracts, unified debugging, secure boundaries

AI-assisted programming	Use models without amplifying insecure, unverifiable, or obsolete code	Specification-guided generation, provenance, testing, proof and review
End-user and accessible programming	Broaden who can create reliable computational artifacts	Visual, conversational, example-based, and domain-native languages
Long-term software preservation	Execute and understand programs after tools and platforms disappear	Executable specifications, emulation, archival toolchains
Governance and ecosystem health	Maintain compatibility and security while evolving languages	Transparent proposal processes and dependency sustainability

8.1 AI-assisted programming changes the interface, not the need for languages

Large language models can generate, translate, explain, and repair code, but generated text still enters a language ecosystem with specific semantics, compilers, libraries, tests, vulnerabilities, and deployment rules. Natural-language prompting does not remove the need for formal or executable representations; it increases the importance of specifications, machine-checkable contracts, sandboxing, provenance, and validation. A likely direction is hybrid programming in which developers express intent through natural language, examples, types, tests, models, and conventional code, while tools synthesize and verify implementations.

8.2 Convergence through intermediate representations and portable runtimes

The future may contain more source languages but fewer execution substrates. LLVM-style intermediate representations, JVM and .NET bytecode, WebAssembly, GPU intermediate forms, database query plans, and hardware synthesis flows allow multiple languages to share optimization and deployment infrastructure. This convergence strengthens multilingual architecture while making semantic boundary design—calling conventions, memory ownership, exceptions, concurrency, and security—an increasingly important research topic.

8.3 Formal methods becoming ordinary engineering

Types, contracts, model checking, abstract interpretation, symbolic execution, proof assistants, and verified compilers are moving from isolated research tools into mainstream development workflows. The central challenge is not only stronger logic but usable feedback: comprehensible errors, incremental checking, automation, integration with package ecosystems, and escape hatches whose risk can be localized and audited.

9. Discussion

The historical record rejects the idea that programming languages evolve by simple replacement. FORTRAN persists because numerical software and compiler expertise accumulated around it. C persists because ABIs, operating systems, embedded targets, and libraries encode decades of investment. Lisp persists through its ideas as much as through its dialects: garbage collection, interactive development, symbolic representation, and macros. ALGOL’s influence survives in block structure and formal grammar. Smalltalk’s influence survives in object systems, live environments, and graphical interfaces. ML and Haskell shaped type inference, algebraic data types, pattern matching, and functional abstraction. Erlang shaped modern thinking about actors and fault tolerance.

The strongest modern designs frequently recombine these inheritances. Rust joins systems-level control with algebraic data types, pattern matching, traits, inference, and ownership. TypeScript overlays a structural type system on a dynamic language ecosystem. Kotlin and Swift combine object orientation with functional idioms, null-safety, inference, and modern tooling. Julia combines interactive dynamic programming with multiple dispatch and aggressive specialization. This pattern supports the paper’s central thesis: innovation is cumulative recombination under new constraints.

A universal “best language” would need simultaneously to maximize safety, expressiveness, predictable performance, portability, simplicity, learnability, ecosystem breadth, metaprogramming, verification, and low implementation cost. These objectives conflict. Language design therefore resembles engineering optimization under constraints. The rational outcome is pluralism, supported by better interoperability, shared tooling, portable intermediate representations, and evidence-based selection.

10. Conclusion

Programming languages are not merely syntactic alternatives for expressing the same algorithms. They embody different theories of abstraction, state, data, time, communication, proof, and resource ownership. A responsible treatment of “all programming languages” must therefore map the design space rather than claim an exhaustive list.

This paper proposed a seven-dimensional model, traced the major historical traditions, compared representative languages, and presented a multi-objective selection method. The review finds no universal winner. Languages succeed when their semantic model, implementation strategy, ecosystem, and governance align with a valuable class of problems. Future progress will likely come from safer resource control, stronger yet usable type and effect systems, verified compilation, heterogeneous hardware support, multilingual interoperability, accessible end-user notation, and AI tools constrained by executable specifications.

The practical conclusion is equally direct: choose languages from requirements, prototypes, evidence, and failure costs; preserve interoperability and exit options; and treat language choice as an architectural decision that should be documented and revisited.

Declarations

Author contribution

SAMUELSON G: Conceptualization; methodology; investigation; synthesis; visualization; writing—original draft; writing—review and editing.

Funding

No external funding was received for this research work.

Conflict of interest

The author declares no known financial or non-financial conflict of interest relevant to this manuscript.

Ethics approval and consent to participate

Not applicable. This review did not involve human participants, animals, clinical records, or personally identifiable data.

Consent for publication

Not applicable.

Data availability

No new empirical dataset was created. The paper synthesizes publicly available standards, specifications, historical documents, and scholarly literature. The comparative tables and figures are author-created syntheses.

Code availability

No experimental software is required to reproduce the narrative synthesis. Figure source files may be archived with the manuscript.

Acknowledgments

The author acknowledges the programming-language researchers, standards bodies, open-source maintainers, archivists, and educators whose work makes comparative scholarship possible.

References

- [1] J. W. Backus et al., “The FORTRAN automatic coding system,” in Proceedings of the Western Joint Computer Conference, 1957, pp. 188–198.
- [2] J. Backus, “The history of FORTRAN I, II, and III,” in History of Programming Languages, R. L. Wexelblat, Ed. New York: Academic Press, 1981, pp. 25–74.
- [3] J. McCarthy, “Recursive functions of symbolic expressions and their computation by machine, Part I,” Communications of the ACM, vol. 3, no. 4, pp. 184–195, 1960, doi: 10.1145/367177.367199.

- [4] P. Naur et al., “Revised report on the algorithmic language ALGOL 60,” *Communications of the ACM*, vol. 6, no. 1, pp. 1–17, 1963, doi: 10.1145/366193.366201.
- [5] CODASYL, COBOL: Initial Specifications for a Common Business Oriented Language. U.S. Department of Defense, 1960.
- [6] K. E. Iverson, *A Programming Language*. New York: Wiley, 1962.
- [7] O.-J. Dahl and K. Nygaard, “SIMULA—an ALGOL-based simulation language,” *Communications of the ACM*, vol. 9, no. 9, pp. 671–678, 1966.
- [8] E. W. Dijkstra, “Go To statement considered harmful,” *Communications of the ACM*, vol. 11, no. 3, pp. 147–148, 1968.
- [9] C. A. R. Hoare, “An axiomatic basis for computer programming,” *Communications of the ACM*, vol. 12, no. 10, pp. 576–580, 1969.
- [10] D. M. Ritchie, “The development of the C language,” *ACM SIGPLAN Notices*, vol. 28, no. 3, pp. 201–208, 1993, doi: 10.1145/154766.155580.
- [11] A. C. Kay, “The early history of Smalltalk,” *ACM SIGPLAN Notices*, vol. 28, no. 3, pp. 69–95, 1993, doi: 10.1145/155360.155364.
- [12] A. Colmerauer and P. Roussel, “The birth of Prolog,” *ACM SIGPLAN Notices*, vol. 28, no. 3, pp. 37–52, 1993.
- [13] R. Milner, “A theory of type polymorphism in programming,” *Journal of Computer and System Sciences*, vol. 17, no. 3, pp. 348–375, 1978.
- [14] L. Cardelli and P. Wegner, “On understanding types, data abstraction, and polymorphism,” *ACM Computing Surveys*, vol. 17, no. 4, pp. 471–523, 1985.
- [15] G. L. Steele Jr. and G. J. Sussman, “The revised report on Scheme: A dialect of Lisp,” *MIT Artificial Intelligence Laboratory Memo 452*, 1978.
- [16] B. Stroustrup, “A history of C++: 1979–1991,” *ACM SIGPLAN Notices*, vol. 28, no. 3, pp. 271–297, 1993.
- [17] J. Gosling et al., *The Java Language Specification, Java SE 25 Edition*. Oracle, 2025.
- [18] Ecma International, *ECMA-262: ECMAScript 2026 Language Specification*, 17th ed., June 2026.
- [19] Python Software Foundation, *The Python Language Reference*, Python 3.14.6 documentation.
- [20] The Go Authors, *The Go Programming Language Specification*, updated 12 January 2026.
- [21] ISO/IEC, *ISO/IEC 9899:2024, Information Technology—Programming Languages—C*, 2024; public working draft N3220.
- [22] The Rust Project Developers, *The Rust Reference*, accessed 13 July 2026.
- [23] R. Jung et al., “RustBelt: Securing the foundations of the Rust programming language,” *Proceedings of the ACM on Programming Languages*, vol. 2, POPL, art. 66, 2018.
- [24] P. Hudak, J. Hughes, S. Peyton Jones, and P. Wadler, “A history of Haskell: Being lazy with class,” in *Proceedings of HOPL III*, 2007, pp. 12-1–12-55, doi: 10.1145/1238844.1238856.
- [25] S. Marlow, Ed., *Haskell 2010 Language Report*. Haskell.org, 2010.
- [26] J. Armstrong, “A history of Erlang,” in *Proceedings of HOPL III*, 2007, pp. 6-1–6-26.

- [27] M. Odersky et al., “An overview of the Scala programming language,” EPFL Technical Report IC/2004/64, 2004.
- [28] G. Bierman, M. Abadi, and M. Torgersen, “Understanding TypeScript,” in ECOOP 2014, LNCS 8586, pp. 257–281, 2014.
- [29] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, “Julia: A fresh approach to numerical computing,” SIAM Review, vol. 59, no. 1, pp. 65–98, 2017.
- [30] N. D. Matsakis and F. S. Klock II, “The Rust language,” ACM SIGAda Ada Letters, vol. 34, no. 3, pp. 103–104, 2014.
- [31] Apple Inc., The Swift Programming Language, official documentation.
- [32] B. C. Pierce, Types and Programming Languages. Cambridge, MA: MIT Press, 2002.
- [33] M. Felleisen, “On the expressive power of programming languages,” Science of Computer Programming, vol. 17, nos. 1–3, pp. 35–75, 1991.
- [34] P. J. Landin, “The next 700 programming languages,” Communications of the ACM, vol. 9, no. 3, pp. 157–166, 1966.
- [35] J. C. Reynolds, “Definitional interpreters for higher-order programming languages,” in Proceedings of the ACM Annual Conference, 1972, pp. 717–740.
- [36] G. D. Plotkin, “Call-by-name, call-by-value and the λ -calculus,” Theoretical Computer Science, vol. 1, no. 2, pp. 125–159, 1975.
- [37] P. Wadler, “The essence of functional programming,” in Proceedings of POPL, 1992, pp. 1–14.
- [38] C. Hewitt, P. Bishop, and R. Steiger, “A universal modular ACTOR formalism for artificial intelligence,” in Proceedings of IJCAI, 1973, pp. 235–245.
- [39] M. Herlihy and J. M. Wing, “Linearizability: A correctness condition for concurrent objects,” ACM Transactions on Programming Languages and Systems, vol. 12, no. 3, pp. 463–492, 1990.
- [40] G. Kiczales et al., “Aspect-oriented programming,” in ECOOP 1997, LNCS 1241, pp. 220–242, 1997.
- [41] M. Mernik, J. Heering, and A. M. Sloane, “When and how to develop domain-specific languages,” ACM Computing Surveys, vol. 37, no. 4, pp. 316–344, 2005.
- [42] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed. Boston: Addison-Wesley, 2006.
- [43] J. C. Mitchell, Foundations for Programming Languages. Cambridge, MA: MIT Press, 1996.
- [44] R. Harper, Practical Foundations for Programming Languages, 2nd ed. Cambridge: Cambridge University Press, 2016.
- [45] J. E. Sammet, Programming Languages: History and Fundamentals. Englewood Cliffs, NJ: Prentice-Hall, 1969.
- [46] T. J. Bergin Jr. and R. G. Gibson Jr., Eds., History of Programming Languages—II. New York: ACM Press/Addison-Wesley, 1996.
- [47] B. W. Kernighan and D. M. Ritchie, The C Programming Language, 2nd ed. Englewood Cliffs, NJ: Prentice Hall, 1988.

- [48] H. Abelson and G. J. Sussman with J. Sussman, Structure and Interpretation of Computer Programs, 2nd ed. Cambridge, MA: MIT Press, 1996.
- [49] ISO/IEC JTC 1/SC 22, international programming-language standardization documents and working-group archives.
- [50] GitHub, “Linguist: language definitions (languages.yml),” maintained source repository, snapshot. The snapshot contained 815 entries: 546 programming, 181 data, 70 markup, and 18 prose.

Appendix A. Compact Classification Checklist

- What is the language’s intended problem domain and unit of abstraction?
- How are syntax and semantics specified, and which behaviors are implementation-defined or undefined?
- Which properties are checked statically, dynamically, or not at all?
- How are memory, files, locks, capabilities, and other resources acquired and released?
- What is the evaluation strategy and implementation path?
- What concurrency and communication mechanisms are built in or conventional?
- How are modules, packages, dependencies, versions, and builds governed?
- What interoperability boundaries exist, and who owns data and errors across them?
- What tools support testing, debugging, profiling, refactoring, security, and deployment?
- What institutional, educational, accessibility, and long-term maintenance constraints affect adoption?